

Base de Datos Relacionales con Python

Uso del gestor de BD PostgreSQL

Instalación y configuración inicial de PostgreSQL en windows 10

Como ya han tenido experiencia en instalación de un Sistema Gestor de BD (MySQL) les dejó el enlace a un vídeo que muestra como realizar la instalación de el gestor de BD PostgreSQL.

<https://techexpert.tips/es/windows-es/instalacion-de-postgresql-en-windows/>

Instalación y configuración inicial OSX

Enlace para seguir el vídeo que muestra la información de como instalar PostgreSQL en MacOS es el siguiente:

<https://progrademia.com/curso/postgresql/3-instalacion-de-postgresql-en-macos/>

Documentación sobre PostgreSQL

A continuación se dejan enlaces para la documentación oficial e información sobre la sintaxis a usar en el SQÑ de PostgreSQL.

Documentación oficial

<https://www.postgresql.org/docs/current/index.html>

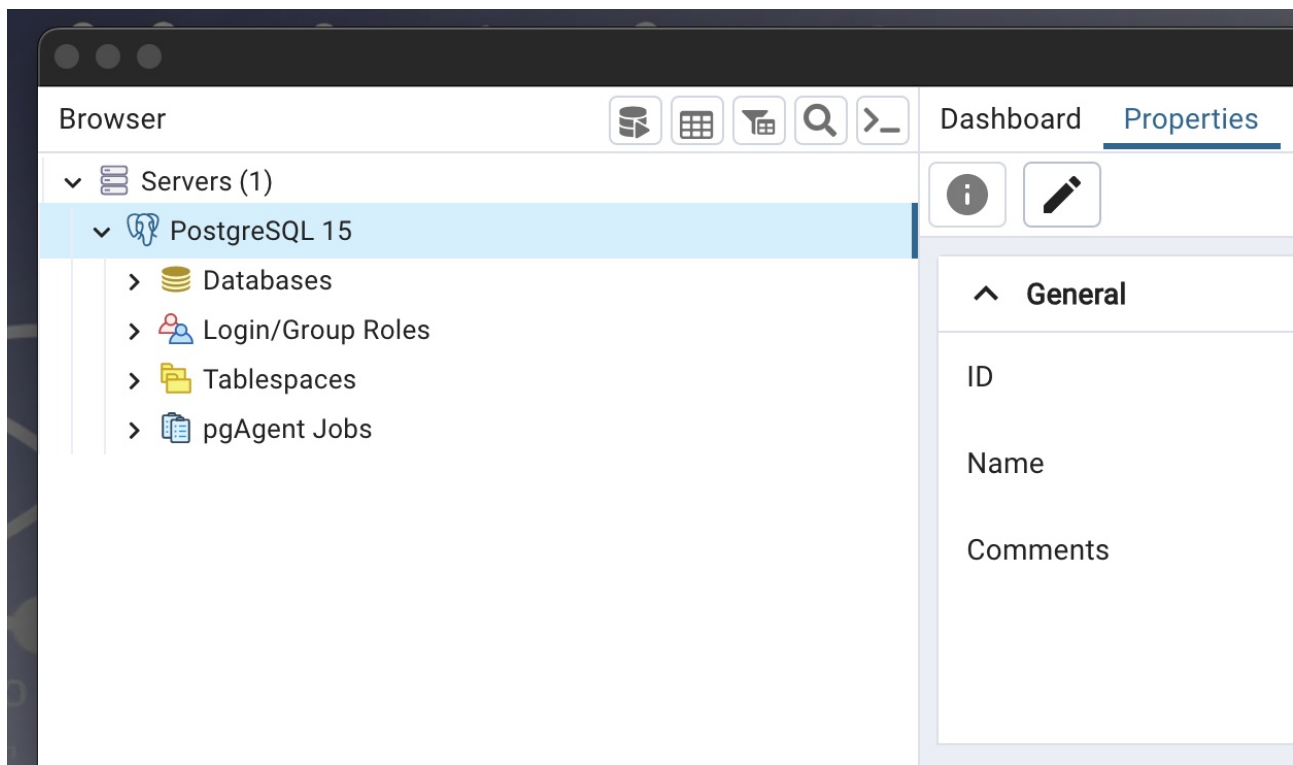
Guía sobre uso y sintaxis de PostgreSQL

<https://www.tutorialspoint.com/postgresql/index.htm>

Una vez que se ha instalado el gestor de BD abrimos pgAdmin4 y verificamos que el servidor esté en funcionamiento. Recordar que los datos iniciales son los siguientes:

Username: postgres

Contraseña: la que hayan puesto durante el proceso de instalación



Creación entorno virtual para usar PostgreSQL e instalación del conector psycopg2

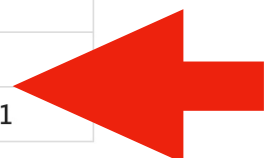
CARPETA DE TRABAJO c:\PRO

En una terminal crear la carpeta anterior y entrar en ella, luego ejecutar el siguiente comando

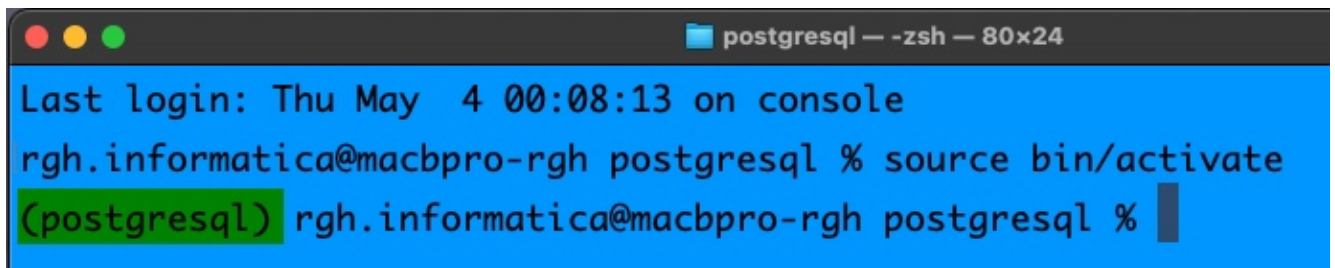
```
> python3 -m venv postgresql
```

Con eso creamos la carpeta postgresql y el entorno virtual situado dentro de ella. Entramos en la carpeta y activamos el entorno, para ello y según el SO y shell usaremos:

Plataforma	Shell	Comando para activar el entorno virtual
POSIX	bash/zsh	\$ source <venv>/bin/activate
	fish	\$ source <venv>/bin/activate.fish
	csh/tcsh	\$ source <venv>/bin/activate.csh
	PowerShell Core	\$ <venv>/bin/Activate.ps1
Windows	cmd.exe	C:\> <venv>\Scripts\activate.bat
	PowerShell	PS C:\> <venv>\Scripts\Activate.ps1



El resultado es el siguiente:

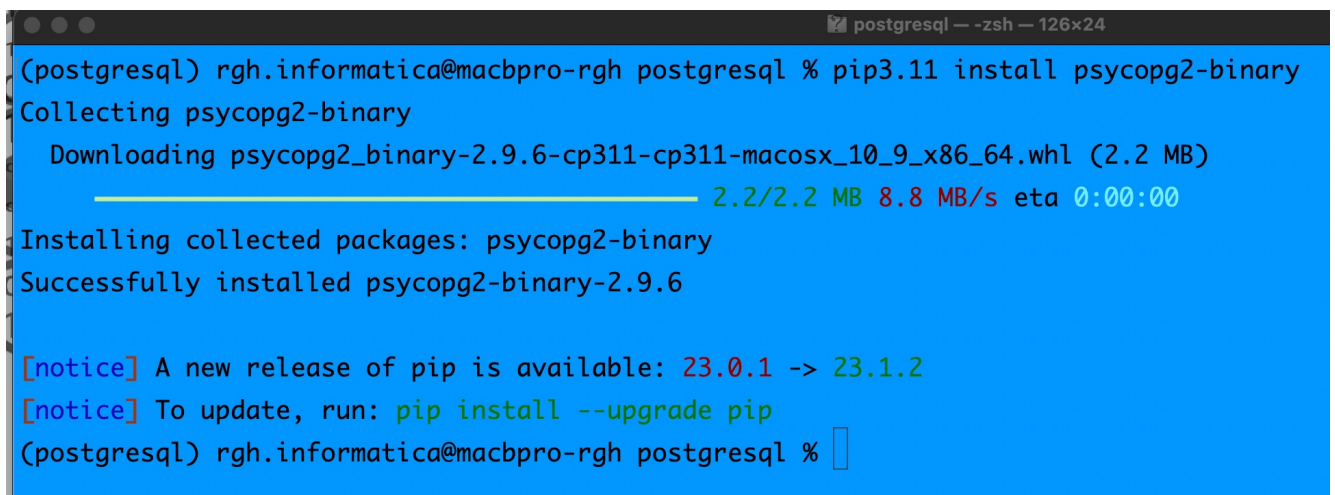


```
postgresql — zsh — 80x24
Last login: Thu May  4 00:08:13 on console
rgh.informatica@macbpro-rgh postgresql % source bin/activate
(postgresql) rgh.informatica@macbpro-rgh postgresql %
```

Notar el indicativo antes del prompt del sistema que muestra que el entorno virtual está activado.

Instalamos ahora el conector Python al gestor PostgreSQL que usaremos en este tema. Este es “**psycopg2**”, para ello usando pip, pip3, pip3.10 o pip3.11 asociado a tu sistema. El comando es el siguiente:

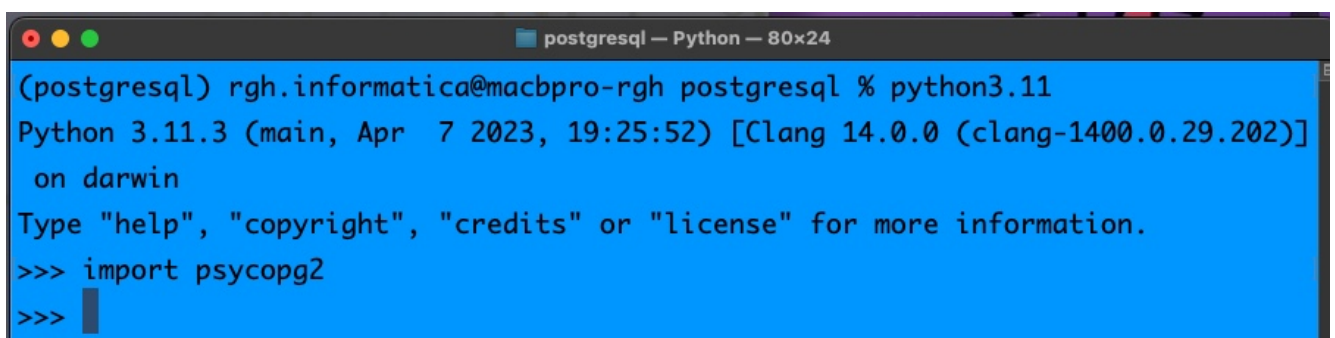
```
>pip install psycopg2-binary
```



```
postgresql — zsh — 126x24
(postgresql) rgh.informatica@macbpro-rgh postgresql % pip3.11 install psycopg2-binary
Collecting psycopg2-binary
  Downloading psycopg2_binary-2.9.6-cp311-cp311-macosx_10_9_x86_64.whl (2.2 MB)
    _____ 2.2/2.2 MB 8.8 MB/s eta 0:00:00
Installing collected packages: psycopg2-binary
Successfully installed psycopg2-binary-2.9.6

[notice] A new release of pip is available: 23.0.1 -> 23.1.2
[notice] To update, run: pip install --upgrade pip
(postgresql) rgh.informatica@macbpro-rgh postgresql %
```

Eso descarga, instala y configura el conector. Para verificar que se ha instalado correctamente:



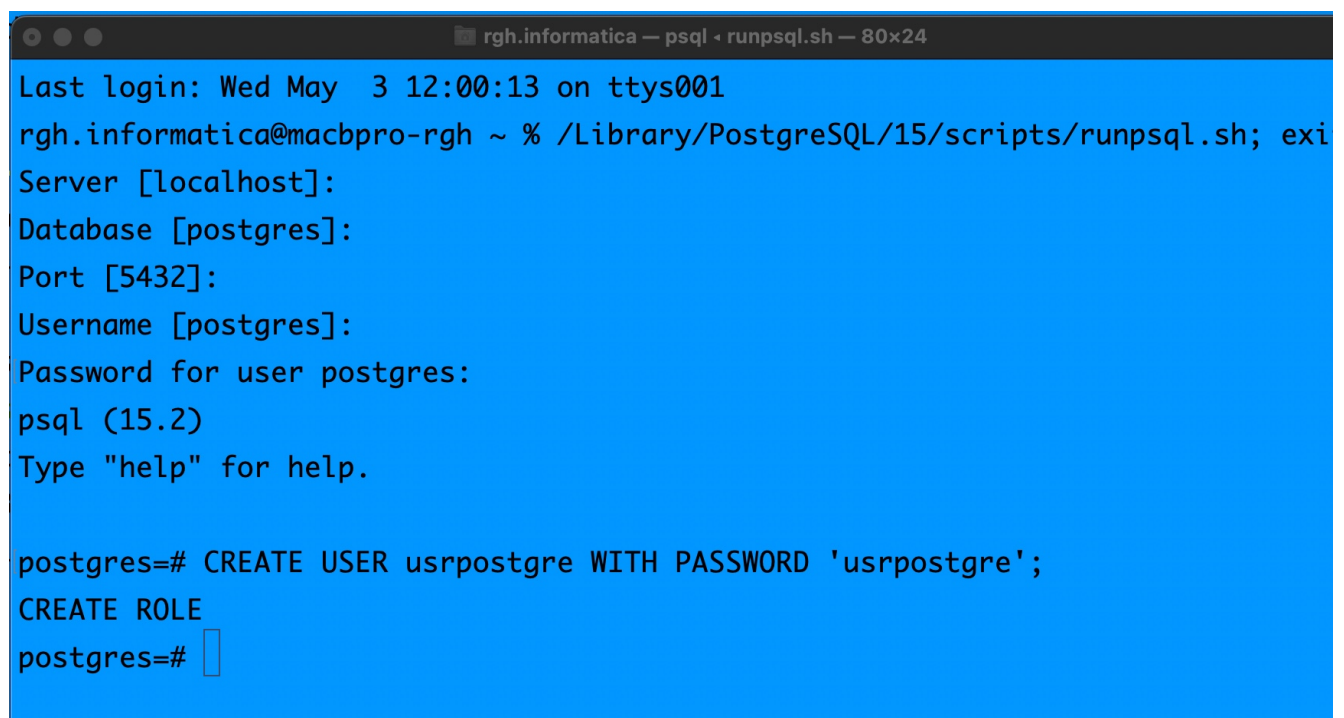
```
postgresql — Python — 80x24
(postgresql) rgh.informatica@macbpro-rgh postgresql % python3.11
Python 3.11.3 (main, Apr  7 2023, 19:25:52) [Clang 14.0.0 (clang-1400.0.29.202)]
on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> import psycopg2
>>>
```

Si no da error, la librería se ha importado correctamente.

Uso del conector psycopg2 para conectar con PostgreSQL

Antes de empezar a usar Python para conectarnos con el servidor, vamos a crear la base de datos de nuestro ejemplo y un usuario al que le daremos todos los privilegios sobre la BD.

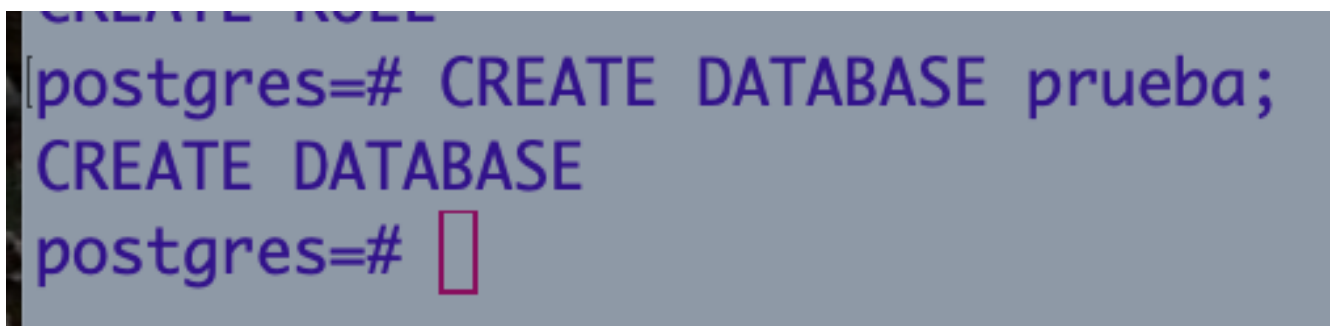
Abrimos una terminal y ejecutamos psql, dentro de la carpeta scripts donde instalaron el programa (o a través de pgAdmin 4) creamos un usuario, en este caso se llamará “**usrpostgre**” y como contraseña “**usrpostgre**”, tal y como se muestra en la figura

A terminal window with a blue background. The title bar reads 'rgh.informatica — psql - runpsql.sh — 80x24'. The terminal shows the following text:

```
Last login: Wed May 3 12:00:13 on ttys001
rgh.informatica@macbpro-rgh ~ % /Library/PostgreSQL/15/scripts/runpsql.sh; exi
Server [localhost]:
Database [postgres]:
Port [5432]:
Username [postgres]:
Password for user postgres:
psql (15.2)
Type "help" for help.

postgres=# CREATE USER usrpostgre WITH PASSWORD 'usrpostgre';
CREATE ROLE
postgres=#
```

A continuación creamos la BD para nuestro ejemplo, ésta se llamará “prueba”.

A terminal window with a grey background. The terminal shows the following text:

```
postgres=# CREATE DATABASE prueba;
CREATE DATABASE
postgres=#
```

Luego le asignamos todos los privilegios al usuario creado sobre la BD.

```
postgres=#  
postgres=# GRANT ALL ON DATABASE prueba TO usrpostgre WITH GRANT OPTION;  
GRANT  
postgres=#
```

Y posteriormente y para evitar el problema que indica a continuación en la nueva versión de PostgreSQL,

The first [comment](#) nailed the most likely reason this is happening. Quoting the [release announcement](#):

PostgreSQL 15 also **revokes the CREATE permission from all users** except a database owner from the **public** (or default) schema.

The reason your fix didn't work is that all actions you took on database `postgres` in regards to user `admin`'s privileges on schema `public` concern only that schema within the database `postgres`. Schema `public` on database `postgres` is not the same schema `public` as the one on newly created `mydb`.

Also, this:

```
GRANT ALL ON DATABASE mydb TO admin;
```

grants privileges on the database itself, not things within the database. `admin` can now drop the database, for example, still without being able to create tables in schema `public`. My guess is that you wanted to make `admin` also the owner of `mydb`, in which case you need to add

```
ALTER DATABASE mydb OWNER TO admin;
```

Ejecutamos el siguiente comando

En el que asigna como propietario de la BD al usuario que hemos creado.

```
postgres=# ALTER DATABASE prueba OWNER TO usrpostgre;  
ALTER DATABASE  
postgres=#
```


Ya está preparada la BD “prueba” para ser usada con el usuario “usrpostgre”.

Veamos como utilizar psycopg2 para conectar con la BD prueba en el servidor. El programa en Python es el siguiente:

```
00-conexion.py X
tema > 00-conexion.py > ...
2  from psycopg2 import Error
3  from getpass import getpass
4
5  try:
6      # Conectamos a la BD prueba con el usuario usrpostgre
7      connection = psycopg2.connect(user="usrpostgre",
8                                     password=getpass("Introduzca la contraseña: "),
9                                     host="localhost",
10                                    port="5432",
11                                    database="prueba")
12
13     print("Información del servidor PostgreSQL")
14     print(f"\n{connection.get_dsn_parameters()}\n")
15     # Creamos un cursor para la ejecución de una consulta
16     cursor = connection.cursor()
17     # creando la consulta
18     consulta = "SELECT version();"
19     # Ejecutando la consulta
20     cursor.execute(consulta)
21     # Obteniendo los resultados
22     record = cursor.fetchone()
23     print(f"Estas conectado - {record}\n")
24
25 except (Exception, Error) as error:
26     print("Error...al intentar conectar con PostgreSQL", error)
27 finally:
28     if (connection):
29         cursor.close()
30         connection.close()
31         print("La conexión con PostgreSQL está cerrada")
32
```

Comentemos los puntos importantes:

1. De esta forma se importa el conector para su uso, también importamos el gestor de errores para trabajar con los errores del gestor.

2. Ejemplo del uso del método “**connect**” para la conexión con el gestor. Recibe los parámetros: “**user**”, el usuario con el que nos vamos a conectar al servidor; “**password**”, la contraseña del usuario; “**host**”, nombre o dirección IP del servidor; “**host**”, el puerto de comunicaciones para realizar la comunicación; “**database**”, la BD a la que queremos conectarnos;. Se suele utilizar un **try** para verificar si ha sido posible la conexión.
3. Con el método **get_dsn_parameters()** obtenemos información acerca del gestor.
4. Con el método “**cursor**” creamos un cursor que nos permitirá ejecutar consultas mediante el método “**execute**”.
5. Para obtener los resultados de la consulta se usa el método “**fetchone**”. Existen otros métodos como “**fetchall**”, o “**fetchmany**”.
6. Con la cláusula “**except**” procesamos el posible error que se produzca al intentar acceder al gestor. En “**error**” está el mensaje de error del sistema.
7. Una vez terminada la ejecución de nuestro algoritmo y antes de terminar el programa, se deben cerrar el cursor y la conexión con el servidor. Esto nos permite liberar recursos.

La ejecución del programa anterior muestra lo siguiente:

```
• (postgres) rgh.informatica@macbpro-rgh postgresql % /Users/rgh.informatica/pro/
n /Users/rgh.informatica/pro/Unidad07/postgresql/programas/01-conexion.py
Información del servidor PostgreSQL

{'user': 'usrpostgre', 'channel_binding': 'prefer', 'dbname': 'prueba', 'host': '
ions': '', 'sslmode': 'prefer', 'sslcompression': '0', 'sslsni': '1', 'ssl_min_pr
encmode': 'prefer', 'krbsrvname': 'postgres', 'target_session_attrs': 'any'}

Estas conectado - ('PostgreSQL 15.2 on x86_64-apple-darwin20.6.0, compiled by App
200.0.32.29), 64-bit',)

La conexión con PostgreSQL está cerrada
○ (postgres) rgh.informatica@macbpro-rgh postgresql % █
```

Como se puede apreciar en la línea 7 del código del programa, la contraseña se puede leer, algo que compromete la seguridad de nuestra BD y del gestor. Una mejora sería utilizar la función **getpass()** de la librería del mismo nombre de la siguiente forma:

```
3 from getpass import getpass
4
5 try:
6     # Conectamos a la BD prueba con el usuario usrpostgre
7     connection = psycopg2.connect(user="usrpostgre",
8                                     password=getpass("Introduzca la contraseña: "),
9                                     host="localhost",
10                                    port="5432",
11                                    database="prueba")
12
```

Ahora al ejecutar el código (01-conexion.py) , se pide al usuario que introduzca la contraseña:

```
• (postgresql) rgh.informatica@macbpro-rgh postgresql % /Users/rgh.informatica/p
Users/rgh.informatica/pro/Unidad07/postgresql/programas/02-crear_tabla.py
Introduzca la contraseña:
Error...al intentar conectar con PostgreSQL relation "mobile" already exists

La conexión con PostgreSQL está cerrada
○ (postgresql) rgh.informatica@macbpro-rgh postgresql %
```

Creación de tablas en una BD en MySQL con Python

El código que permite crear una tabla en una BD es el siguiente:


```
programas > 02-crear_tabla.py > ...
1  import psycopg2
2  from psycopg2 import Error
3  from getpass import getpass
4
5  try:
6      # Conectamos a la BD prueba con el usuario usrpostgres
7      connection = psycopg2.connect(user="usrpostgres",
8                                     password=getpass("Introduzca la contraseña: "),
9                                     host="localhost",
10                                    port="5432",
11                                    database="prueba")
12
13     cursor = connection.cursor()
14     query = '''CREATE TABLE mobile
15              (ID INT PRIMARY KEY NOT NULL,
16               MODEL TEXT NOT NULL,
17               PRICE REAL);'''
18
19     # Ejecutando la consulta
20     cursor.execute(query)
21     # consolidamos la modificación de la BD
22     connection.commit()
23     print("La tabla mobile has sido creada satisfactoriamente")
24 except (Exception, Error) as error:
25     print("Error...al intentar conectar con PostgreSQL", error)
26 finally:
27     if (connection):
28         # cursor.close()
29         connection.close()
30         print("La conexión con PostgreSQL está cerrada")
31
```

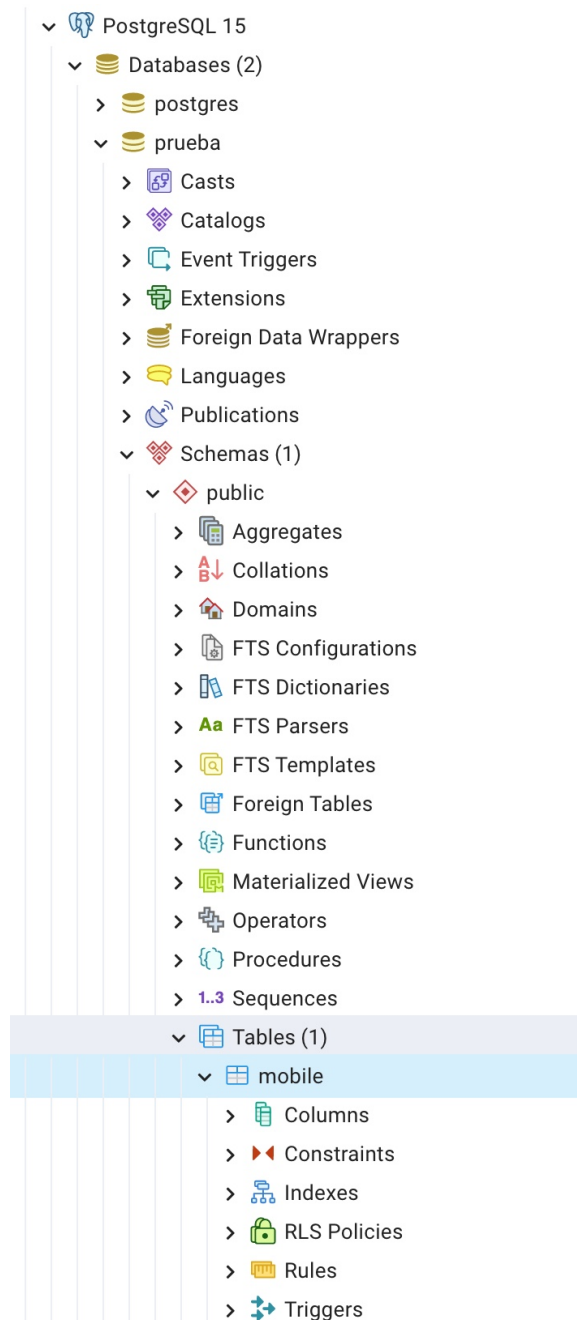
Comentarios de los apartados:

1. Se debe crear un string con el script en SQL que tiene la instrucción y parámetros necesarios que permitan crear la tabla. Este string suele realizarse con “`'''`” para poder escribirlo sin tener en cuenta las tabulaciones de Python”.
2. Se crea un cursor (línea 17) que será el medio con el que se ejecute el script SQL creado. El script SQL debe estar bien formado, porque sino el servidor dará error al ejecutar la instrucción.

El resultado de la ejecución del programa es el siguiente:

```
• (postgresql) rgh.informatica@macbpro-rgh postgresql % /Users/rgh.informatica/pro/U
Users/rgh.informatica/pro/Unidad07/postgresql/programas/02-crear_tabla.py
Introduzca la contraseña:
La tabla mobile has sido creada satisfactoriamente
La conexión con PostgreSQL está cerrada
○ (postgresql) rgh.informatica@macbpro-rgh postgresql %
```

Y con pgAdmin 4, podemos verificar la creación de la tabla.



Proceso de datos en una tabla PostgreSQL con Python

El esquema de proceso para realizar operaciones en una tabla es el siguiente:

Los pasos a realizar para insertar, modificar o borrar datos en una tabla son los siguientes:

1. Conectarse a PostgreSQL desde Python.
2. Definir una consulta SQL para insertar, modificar, borrar datos en la tabla.
3. Obtener objeto cursor de la conexión.
4. Ejecutar la consulta utilizando el método **execute()**
5. Hacer persistentes los datos en la tabla, uso de **commit()**
6. **(Opcional)** Verificar el resultado utilizando la consulta SQL SELECT
7. Cerrar el objeto cursor y el objeto de conexión a la base de datos

Inserción de datos en una tabla PostgreSQL con Python

Veamos un ejemplo de código que permita insertar una fila de datos en la tabla “**mobile**” de nuestra BD.

```
programas > 03-operaciones_crud-insertar.py > ...
1  import psycopg2
2  from psycopg2 import Error
3
4  try:
5      # Conectamos a la BD prueba con el usuario usrpostgres
6      connection = psycopg2.connect(user="usrpostgres",
7                                     password="usrpostgres",
8                                     host="localhost",
9                                     port="5432",
10                                    database="prueba")
11
12     cursor = connection.cursor()
13
14     1 # inserción de datos en la tabla mobile
15     query_insertar = """INSERT INTO mobile (ID, MODEL, PRICE)
16                        VALUES (1, 'Iphone12', 1100)"""
17
18     2 cursor.execute(query_insertar)
19     connection.commit()
20     print("1 Registro insertado satisfactoriamente")
21
22     # obtener resultado de inserción (ejecución select de la tabla)
23     3 cursor.execute("SELECT * from mobile")
24     registro = cursor.fetchall()
25     print("Result ", registro)
26 except (Exception, Error) as error:
27     print("Error...al intentar conectar con PostgreSQL", error)
28 finally:
29     if (connection):
30         # cursor.close()
31         connection.close()
32         print("La conexión con PostgreSQL está cerrada")
```

Veamos los apartados destacados:

1. Se crea el string con el código SQL que permite insertar un registro en la tabla y el cursor para su ejecución.
2. Se ejecuta la consulta y se consolidan los datos.

3. (Opcional) Se realiza una consulta de select para verificar que el registro se añadió a la tabla.

El resultado de la ejecución y el contenido de la tabla son los siguientes:

```
• (postgresql) rgh.informatica@macbpro-rgh postgresql % /Use
Users/rgh.informatica/pro/Unidad07/postgresql/programas/03
1 Registro insertado satisfactoriamente
Result [(1, 'Iphone12', 1100.0)]
La conexión con PostgreSQL está cerrada
○ (postgresql) rgh.informatica@macbpro-rgh postgresql %
```

>_ Dashboard Properties SQL Statistics Depen

```
psql (15.2)
Type "help" for help.

prueba=# select * from mobile;
 id |  model   | price 
----+-----+-----
  1 | Iphone12 |  1100 
(1 row)

prueba=#
```


Modificación de datos en una tabla PostgreSQL con Python

El proceso es similar al de la inserción de datos el código es el siguiente:

```
programas > 04-operaciones_crud-modificar.py > ...
1  import psycopg2
2  from psycopg2 import Error
3
4  try:
5      # Conectamos a la BD prueba con el usuario usrpostgres
6      connection = psycopg2.connect(user="usrpostgres",
7                                     password="usrpostgres",
8                                     host="localhost",
9                                     port="5432",
10                                    database="prueba")
11
12     cursor = connection.cursor()
13
14     # modificación de datos en la tabla mobile
15     query_insertar = """UPDATE mobile SET price = 1500 WHERE id = 1"""
16     cursor.execute(query_insertar)
17     connection.commit()
18     count = cursor.rowcount
19     print(f"{count} Registro modificado satisfactoriamente")
20
21     # obtener resultado de inserción (ejecución select de la tabla)
22     cursor.execute("SELECT * from mobile")
23     registro = cursor.fetchall()
24     print("Result ", registro)
25 except (Exception, Error) as error:
26     print("Error...al intentar conectar con PostgreSQL", error)
27 finally:
28     if (connection):
29         # cursor.close()
30         connection.close()
31         print("La conexión con PostgreSQL está cerrada")
32
```

La ejecución del programa muestra los siguientes resultados:

```
• (postgresql) rgh.informatica@macbpro-rgh postgresql % /Users/rgh.informatica/pro/h.informatica/pro/Unidad07/postgresql/programas/04-operaciones_crud-modificar.py
1 Registro modificado satisfactoriamente
Result  [(1, 'Iphone12', 1500.0)]
La conexión con PostgreSQL está cerrada
○ (postgresql) rgh.informatica@macbpro-rgh postgresql %
```

Data Output		Messages	Notifications
       			
	id [PK] integer 	model text 	price real 
1	1	Iphone12	1500

Eliminación de datos en una tabla MySQL con Python

El proceso es similar al de la inserción de datos el código es el siguiente:

```
programas > 05-operaciones_crud-eliminar.py > ...
1  import psycopg2
2  from psycopg2 import Error
3
4  try:
5      # Conectamos a la BD prueba con el usuario usrpostgre
6      connection = psycopg2.connect(user="usrpostgre",
7                                     password="usrpostgre",
8                                     host="localhost",
9                                     port="5432",
10                                    database="prueba")
11
12     cursor = connection.cursor()
13
14     # eliminando un registro de la BD
15     query_borrar = """DELETE FROM mobile WHERE id = 1"""
16     cursor.execute(query_borrar)
17     connection.commit()
18     count = cursor.rowcount
19     print(f"{count} Registro eliminado satisfactoriamente")
20
21     # obtener resultado de inserción (ejecución select de la tabla)
22     cursor.execute("SELECT * from mobile")
23     registro = cursor.fetchall()
24     print("Result ", registro)
25 except (Exception, Error) as error:
26     print("Error...al intentar conectar con PostgreSQL", error)
27 finally:
28     if (connection):
29         # cursor.close()
30         connection.close()
31         print("La conexión con PostgreSQL está cerrada")
32
```

La ejecución del programa muestra los siguientes resultados:

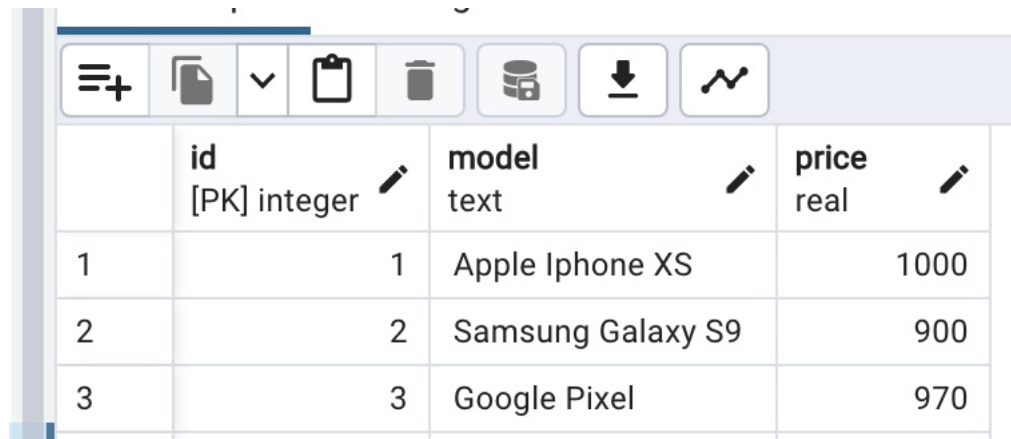
```
• rgh.informatica@macbpro-rgh postgresql % source /Users/rgh
• (postgresql) rgh.informatica@macbpro-rgh postgresql % /Use
h.informatica/pro/Unidad07/postgresql/programas/05-operaci
1 Registro eliminado satisfactoriamente
Result  []
La conexión con PostgreSQL está cerrada
○ (postgresql) rgh.informatica@macbpro-rgh postgresql %
```

Data Output		Messages		Notifications			
							
	id [PK] integer 	model text 	price real 				

Insertar, Modificar y Borrar multiples filas con executemany()

Mostraremos un ejemplo de como insertar múltiples datos en una sola instrucción

Estado de la tabla Mobile antes de la ejecución



	id [PK] integer	model text	price real
1	1	Apple Iphone XS	1000
2	2	Samsung Galaxy S9	900
3	3	Google Pixel	970

El programa es el siguiente

programas > 06-insercion-multiple.py > ...

```
1  import psycopg2
2  from psycopg2 import Error
3
4  try:
5      # Conectamos a la BD prueba con el usuario usrpostgre
6      connection = psycopg2.connect(user="usrpostgre",
7                                     password="usrpostgre",
8                                     host="localhost",
9                                     port="5432",
10                                    database="prueba")
11
12     cursor = connection.cursor()
13
14     # inserción de datos en la tabla mobile
15     query_insertar = """ INSERT INTO mobile (id, model, price)
16                           VALUES (%s,%s,%s);"""
17     records = [(4, 'LG', 800), (5, 'One Plus 6', 950)]
18     result = cursor.executemany(query_insertar, records)
19     connection.commit()
20     count = cursor.rowcount
21     print(f"{count} Registros insertados satisfactoriamente")
22 except (Exception, Error) as error:
23     print("Error...al intentar conectar con PostgreSQL", error)
24 finally:
25     if (connection):
26         # cursor.close()
27         connection.close()
28         print("La conexión con PostgreSQL está cerrada")
29
```


La ejecución del programa muestra los siguientes resultados

```
• rgh.informatica@macbpro-rgh postgresql % source /Users/rgh.informatica/pro/Unidad07/postgresql/bin/activate
• (postgresql) rgh.informatica@macbpro-rgh postgresql % /Users/rgh.informatica/pro/Unidad07/postgresql/bin/python /Users/rgh.informatica/pro/Unidad07/postgresql/programas/06-insercion-multiple.py
2 Registros insertados satisfactoriamente
La conexión con PostgreSQL está cerrada
○ (postgresql) rgh.informatica@macbpro-rgh postgresql %
```

Data Output				Messages	Notifications
	id [PK] integer	model text	price real		
1	1	Apple Iphone XS	1000		
2	2	Samsung Galaxy S9	900		
3	3	Google Pixel	970		
4	4	LG	800		
5	5	One Plus 6	950		

Modificar multiples filas con executemany()

Analicemos el programa siguiente que modifica múltiples registros en la tabla.

```
programas > 07-modificacion-multiple.py > ...
1  import psycopg2
2  from psycopg2 import Error
3
4  try:
5      # Conectamos a la BD prueba con el usuario usrpostgre
6      connection = psycopg2.connect(user="usrpostgre",
7                                     password="usrpostgre",
8                                     host="localhost",
9                                     port="5432",
10                                    database="prueba")
11
12     cursor = connection.cursor()
13
14     # modificación de datos en la tabla mobile
15     query_insertar = """Update mobile set price = %s where id = %s"""
16     records = [(750, 4), (950, 5)]
17     result = cursor.executemany(query_insertar, records)
18     connection.commit()
19     count = cursor.rowcount
20     print(f"{count} Registros modificados satisfactoriamente")
21 except (Exception, Error) as error:
22     print("Error...al intentar conectar con PostgreSQL", error)
23 finally:
24     if (connection):
25         # cursor.close()
26         connection.close()
27         print("La conexión con PostgreSQL está cerrada")
28
```

La ejecución del programa muestra los siguientes resultados

```
• (postgresql) rgh.informatica@macbpro-rgh postgresql % /Users/
ostgresql/bin/python /Users/rgh.informatica/pro/Unidad07/pos
on-multiple.py
2 Registros modificados satisfactoriamente
La conexión con PostgreSQL está cerrada
○ (postgresql) rgh.informatica@macbpro-rgh postgresql %
```

Data Output				Messages	Notifications
	id [PK] integer	model text	price real		
1	1	Apple Iphone XS	1000		
2	2	Samsung Galaxy S9	900		
3	3	Google Pixel	970		
4	4	LG	750		
5	5	One Plus 6	950		

Borrar multiples filas con executemany()

Analicemos el programa siguiente que modifica múltiples registros en la tabla.

programas > 08-borrado-multiple.py > ...

```
1  import psycopg2
2  from psycopg2 import Error
3
4  try:
5      # Conectamos a la BD prueba con el usuario usrpostgre
6      connection = psycopg2.connect(user="usrpostgre",
7                                     password="usrpostgre",
8                                     host="localhost",
9                                     port="5432",
10                                    database="prueba")
11
12     cursor = connection.cursor()
13
14     # borrado de datos en la tabla mobile
15     query_insertar = "Delete from mobile where id = %s"
16     records = [(5,), (4,), (3,)]
17     result = cursor.executemany(query_insertar, records)
18     connection.commit()
19     count = cursor.rowcount
20     print(f"{count} Registros borrados satisfactoriamente")
21 except (Exception, Error) as error:
22     print("Error...al intentar conectar con PostgreSQL", error)
23 finally:
24     if (connection):
25         cursor.close()
26         connection.close()
27         print("La conexión con PostgreSQL está cerrada")
28
```

La ejecución del programa muestra los siguientes resultados

```
• (postgresql) rgh.informatica@macbpro-rgh postgresql % /Users/rgh.informatica/pro/Unidad07/po
ostgresql/bin/python /Users/rgh.informatica/pro/Unidad07/po
ltiple.py
3 Registros borrados satisfactoriamente
La conexión con PostgreSQL está cerrada
○ (postgresql) rgh.informatica@macbpro-rgh postgresql %
```

Data Output				Messages	Notifications
	id [PK] integer	model text	price real		
1	1	Apple Iphone XS	1000		
2	2	Samsung Galaxy S9	900		

La correspondencia entre los tipos Python y PostgreSQL

Hay un mapeo por defecto especificado para convertir tipos Python en equivalentes PostgreSQL, y viceversa. Cada vez que ejecute una consulta PostgreSQL utilizando Python siguiente tabla es utilizada por psycopg2 para devolver el resultado en forma de objetos Python.

Python	PostgreSQL
None	NULL
bool	bool
float	real or double
int	smallint integer bigint
Decimal	numeric
str	varchar text
date	date
time	time timetz
datetime	timestamp timestampz
timedelta	interval
list	ARRAY
tuple	Composite types IN syntax
dict	hstore

Constantes y conversión numérica

Cuando intentas insertar valores None y booleanos de Python como True y False en PostgreSQL, se convierten en los literales SQL adecuados. Lo mismo ocurre con los tipos numéricos de Python. Se convierten en tipos PostgreSQL equivalentes.

Por ejemplo, cuando ejecuta una consulta de inserción, los objetos numéricos de Python como int, long, float, Decimal se convierten en una representación numérica de PostgreSQL. Cuando usted lee de la tabla PostgreSQL, los tipos enteros son convertidos en un int, los tipos de punto flotante son convertidos en un float, numeric/Decimal son convertidos en Decimal.

Trabajando con fecha y hora PostgreSQL en Python

Esta sección demostrará cómo trabajar con los tipos de datos de fecha y hora de PostgreSQL en Python y viceversa. La mayoría de las veces, trabajamos con datos de fecha y hora. Insertamos fecha y hora en la tabla y también leemos de ella en nuestra aplicación cuando es necesario.

En un escenario habitual, cuando ejecutas la consulta de inserción con el objeto datetime, el módulo psycopg2 de Python lo convierte a un formato de fecha y hora de PostgreSQL para insertarlo en la tabla.

Y cuando ejecutas una consulta SELECT desde Python para leer valores timestamp de la tabla PostgreSQL, el módulo psycopg2 lo convierte en un objeto datetime.

Estamos utilizando la tabla "Item" para esta demostración. Por favor copie y ejecute la siguiente consulta en su herramienta de consulta PostgreSQL para tener los datos adecuados para esta operación.

Código para crear la tabla "ítem"

```

programas > 09-crear_tabla-datetime.py > ...
1  import psycopg2
2  from psycopg2 import Error
3  from getpass import getpass
4
5  try:
6      # Conectamos a la BD prueba con el usuario usrpostgre
7      connection = psycopg2.connect(user="usrpostgre",
8                                     password=getpass("Introduzca la contraseña: "),
9                                     host="localhost",
10                                    port="5432",
11                                    database="prueba")
12
13     query = """CREATE TABLE item (item_id serial NOT NULL PRIMARY KEY,
14                                   item_name VARCHAR (100) NOT NULL,
15                                   purchase_time timestamp NOT NULL,
16                                   price INTEGER NOT NULL);"""
17
18     # Creamos el cursor y ejecutamos la consulta
19     cursor = connection.cursor()
20     cursor.execute(query)
21     # consolidamos la modificación de la BD
22     connection.commit()
23     print("La tabla mobile has sido creada satisfactoriamente")
24 except (Exception, Error) as error:
25     print("Error...al intentar conectar con PostgreSQL", error)
26 finally:
27     if (connection):
28         cursor.close()
29         connection.close()
30     print("La conexión con PostgreSQL está cerrada")

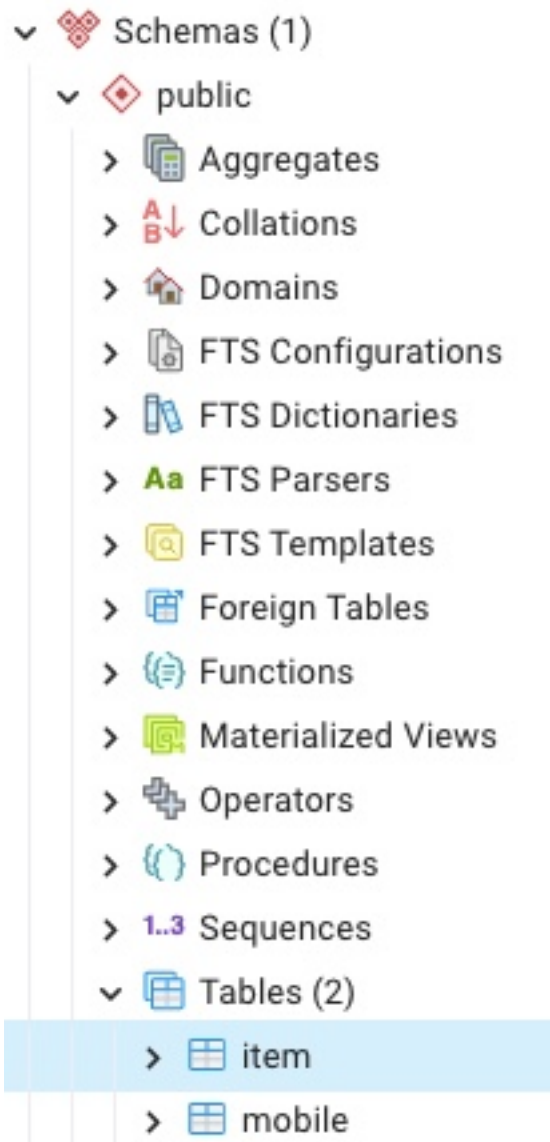
```

El resultado de la ejecución es el siguiente:

```

• (postgresql) rgh.informatica@macbpro-rgh postgresql % /
python /Users/rgh.informatica/pro/Unidad07/postgresql/p
Introduzca la contraseña:
La tabla mobile has sido creada satisfactoriamente
La conexión con PostgreSQL está cerrada
○ (postgresql) rgh.informatica@macbpro-rgh postgresql %

```



Vamos a entender este escenario con un ejemplo sencillo. Aquí leeremos la columna `purchase_time` de la tabla PostgreSQL y la convertiremos en un objeto `datetime` de Python.

El programa es el siguiente:

```

programas > 10-usar-datetime.py > ...
1  import psycopg2
2  from psycopg2 import Error
3  from getpass import getpass
4  import datetime
5
6  try:
7      # Conectamos a la BD prueba con el usuario usrpostgre
8      connection = psycopg2.connect(user="usrpostgre",
9                                     password=getpass("Introduzca la contraseña: "),
10                                    host="localhost",
11                                    port="5432",
12                                    database="prueba")
13
14      query = """INSERT INTO item (item_Id, item_name, purchase_time, price)
15                  VALUES (%s, %s, %s, %s);"""
16      item_purchase_time = datetime.datetime.now()
17      item_tuple = (12, "Keyboard", item_purchase_time, 150)
18      # Creamos el cursor y ejecutamos la consulta
19      cursor = connection.cursor()
20      cursor.execute(query, item_tuple)
21      # consolidamos la modificación de la BD
22      connection.commit()
23      count = cursor.rowcount
24      print(f"{count} Registro insertado satisfactoriamente")
25
26      # Leer PostgreSQL timestamp en Python datetime
27      cursor.execute("SELECT purchase_time from item where item_id = 12")
28      purchase_datetime = cursor.fetchone()
29      print("Item Purchase date is ", purchase_datetime[0].date())
30      print("Item Purchase time is ", purchase_datetime[0].time())
31
32  except (Exception, Error) as error:
33      print("Error...al intentar conectar con PostgreSQL", error)
34  finally:
35      if (connection):
36          cursor.close()
37          connection.close()
38          print("La conexión con PostgreSQL está cerrada")
39

```

La ejecución muestra los siguiente resultados

```

• (postgresql) rgh.informatica@macbpro-rgh postgresql % /U
python /Users/rgh.informatica/pro/Unidad07/postgresql/pr
Introduzca la contraseña:
1 Registro insertado satisfactoriamente
Item Purchase date is    2023-05-10
Item Purchase time is    01:38:44.209349
La conexión con PostgreSQL está cerrada
○ (postgresql) rgh.informatica@macbpro-rgh postgresql % █

```

Data Output Messages Notifications				
       				
	item_id [PK] integer	item_name character varying (100)	purchase_time timestamp without time zone	price integer
1	12	Keyboard	2023-05-10 01:38:44.209349	150

Métodos avanzado de uso de Python para trabajar BD PostgreSQL

Llamada a Función PostgreSQL y Procedimiento Almacenado desde Python

En estos temas avanzados se les deja enlaces a documentación en inglés para que los revisen y sepan como trabajar con ellos

<https://pynative.com/python-execute-postgresql-stored-procedure-and-function/>

Python PostgreSQL Gestión de transacciones utilizando Commit y Rollback

<https://pynative.com/python-postgresql-transaction-management-using-commit-and-rollback/>